

1. Make Github Repo
2. Make virtual environment and activate python environment

```
python -m venv .venv
```

```
.venv\Scripts\activate
```

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Valin\Downloads\cloud-native-devops-platform> python --version
Python 3.13.7
PS C:\Users\Valin\Downloads\cloud-native-devops-platform> pip --version
pip 25.3 from C:\Users\Valin\AppData\Roaming\Python\Python313\site-packages\pip (python 3.13)
PS C:\Users\Valin\Downloads\cloud-native-devops-platform> python -m venv .venv
PS C:\Users\Valin\Downloads\cloud-native-devops-platform> .venv\Scripts\activate
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform>
```

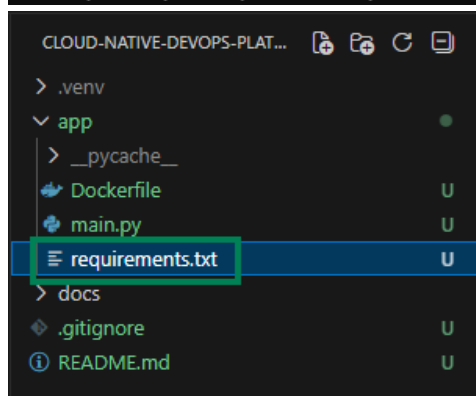
3. Install FastAPI

```
pip install fastapi uvicorn
```

```
pip freeze > requirements.txt -- "Record all Python packages currently installed
in this environment in a file called requirements.txt."
```

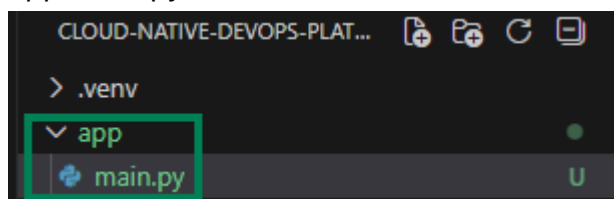
```
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> pip install fastapi uvicorn
Collecting fastapi
  Downloading fastapi-0.141.1-py3-none-any.whl.metadata (27 kB)
Collecting uvicorn
  Downloading uvicorn-0.53.0-py3-none-any.whl.metadata (6.6 kB)
```

```
PS C:\Users\Valin\Downloads\cloud-native-devops-platform\app> pip freeze > requirements.txt
PS C:\Users\Valin\Downloads\cloud-native-devops-platform\app>
```



4. Create the first application

```
app/main.py
```



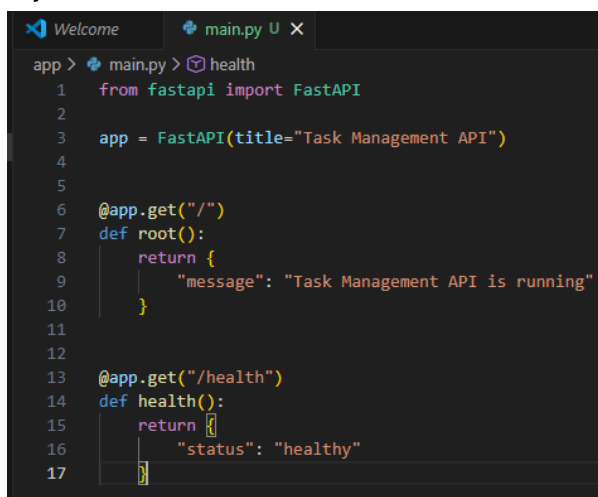
Content:

```
from fastapi import FastAPI
```

```
app = FastAPI(title="Task Management API")
```

```
@app.get("/")
def root():
    return {
        "message": "Task Management API is running"
    }
```

```
@app.get("/health")
def health():
    return {
        "status": "healthy"
    }
```

A screenshot of a code editor window with a dark theme. The window has two tabs: 'Welcome' and 'main.py U X'. The 'main.py' tab is active, showing the code from the previous blocks. The code is as follows:

```
app > main.py > health
1  from fastapi import FastAPI
2
3  app = FastAPI(title="Task Management API")
4
5
6  @app.get("/")
7  def root():
8      return {
9          "message": "Task Management API is running"
10     }
11
12
13 @app.get("/health")
14 def health():
15     return {
16         "status": "healthy"
17     }
```

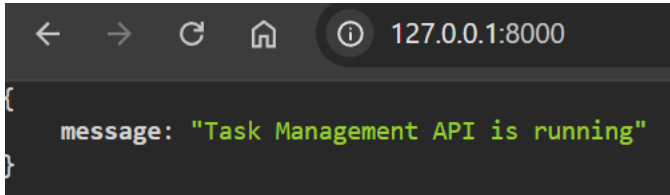
uvicorn main:app --reload --- (first, move to folder app)

```
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> cd
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> dir

Directory: C:\Users\Valin\Downloads\cloud-native-devops-platform

Mode                LastWriteTime         Length Name
----                -
d-----          9/14/2026   4:55 PM             .venv
d-----          9/14/2026   5:11 PM             app
d-----          9/14/2026   4:41 PM             docs
-a-----          9/14/2026   4:44 PM              0 .gitignore
-a-----          9/14/2026   4:43 PM              0 README.md
-a-----          9/14/2026   5:05 PM          496 requirements.txt

(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> cd app
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform\app> uvicorn main:app --reload
INFO: Will watch for changes in these directories: ['C:\\Users\\Valin\\Downloads\\cloud-native-devops-platform\\app']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [34484] using StatReload
INFO: Started server process [39276]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:64570 - "GET / HTTP/1.1" 200 OK
INFO: 127.0.0.1:64570 - "GET /favicon.ico HTTP/1.1" 404 Not Found
```



5. Try /health

Open <http://127.0.0.1:8000/health>

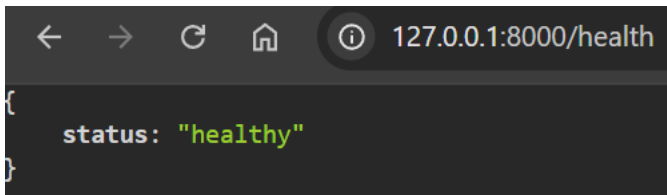
This is actually simple, but this endpoint will be very useful for Kubernetes later.

Kubernetes can then ask: "Is your application still alive?"

Application:

/status → healthy

If it's not healthy, Kubernetes can take action.



6. Look at FastAPI Swagger

Open <http://127.0.0.1:8000/docs>

We will see interactive API documentation



Task Management API 0.1.0 OAS 3.1

/openapi.json

default

GET

/ Root

GET

/health Health

Try it out → Execute



GET /health Health

Parameters

No parameters

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/health' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/health
```

Server response

Code	Details
200	Response body <pre>{ "status": "healthy" }</pre> Response headers <pre>content-length: 20 content-type: application/json date: Mon, 14 Sep 2026 15:27:56 GMT server: uvicorn</pre>

Responses

Code	Description	Links
------	-------------	-------

7. Create the Task API.

Now we start building the actual application.

Change the main.py content become:

from fastapi import FastAPI

from pydantic import BaseModel

```
app = FastAPI(title="Task Management API")
```

```
class Task(BaseModel):
```

```
    title: str
```

```
    completed: bool = False
```

```
tasks = []
```

```
@app.get("/")
def root():
    return {
        "message": "Task Management API is running"
    }
```

```
@app.get("/health")
def health():
    return {
        "status": "healthy"
    }
```

```
@app.get("/tasks")
def get_tasks():
    return tasks
```

```
@app.post("/tasks")
def create_task(task: Task):
    new_task = {
        "id": len(tasks) + 1,
        "title": task.title,
        "completed": task.completed
    }
```

```
tasks.append(new_task)
```

```
return new_task
```

```
Welcome  main.py U x
app > main.py > health
5
6
7 class Task(BaseModel):
8     title: str
9     completed: bool = False
10
11
12 tasks = []
13
14
15 @app.get("/")
16 def root():
17     return {
18         "message": "Task Management API is running"
19     }
20
21
22 @app.get("/health")
23 def health():
24     return {
25         "status": "healthy"
26     }
27
28
29 @app.get("/tasks")
30 def get_tasks():
31     return tasks
32
33
34 @app.post("/tasks")
35 def create_task(task: Task):
36     new_task = {
37         "id": len(tasks) + 1,
38         "title": task.title,
39         "completed": task.completed
40     }
41
42     tasks.append(new_task)
43
44     return new_task
```

Restart the server if needed

Open <http://127.0.0.1:8000/docs>

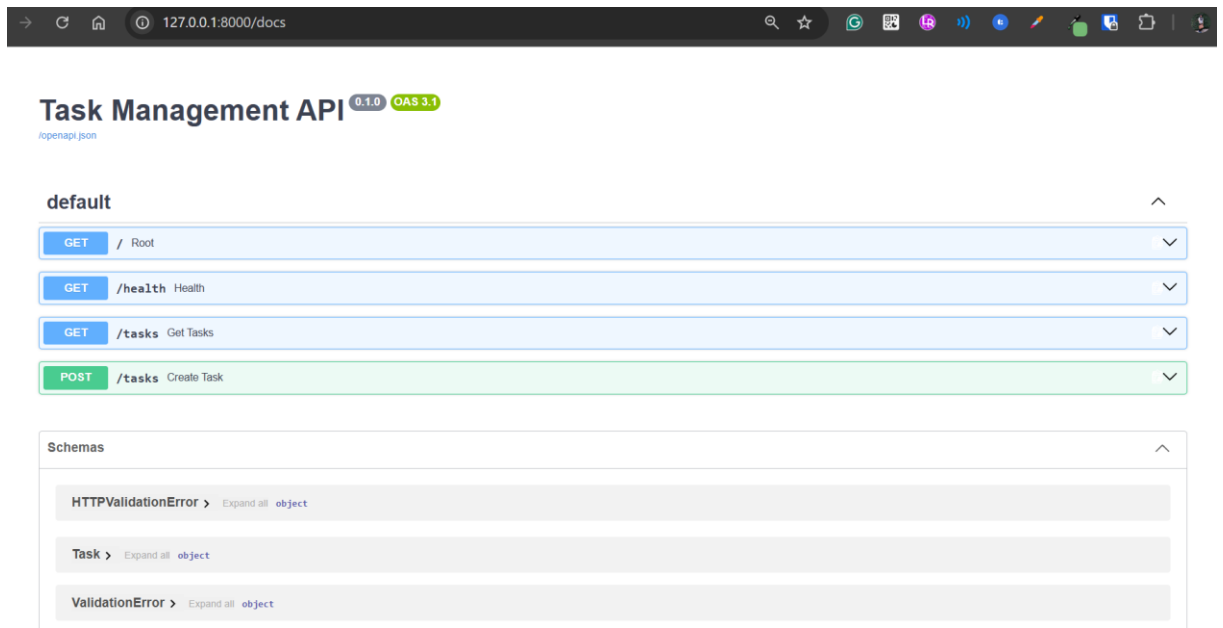
Now you will see:

GET /

GET /health

GET /tasks

POST /tasks

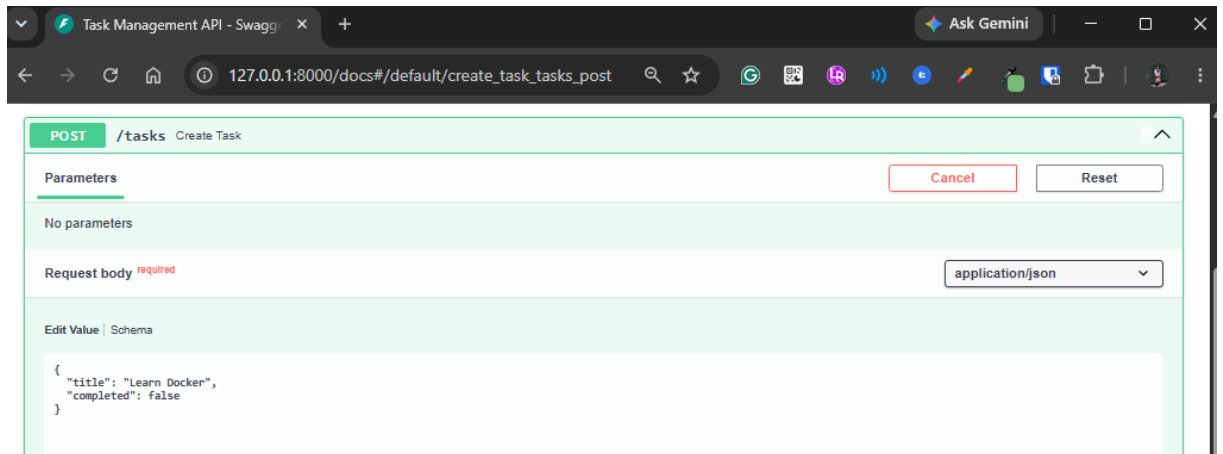


Now, let's play on the Swagger:

POST /tasks

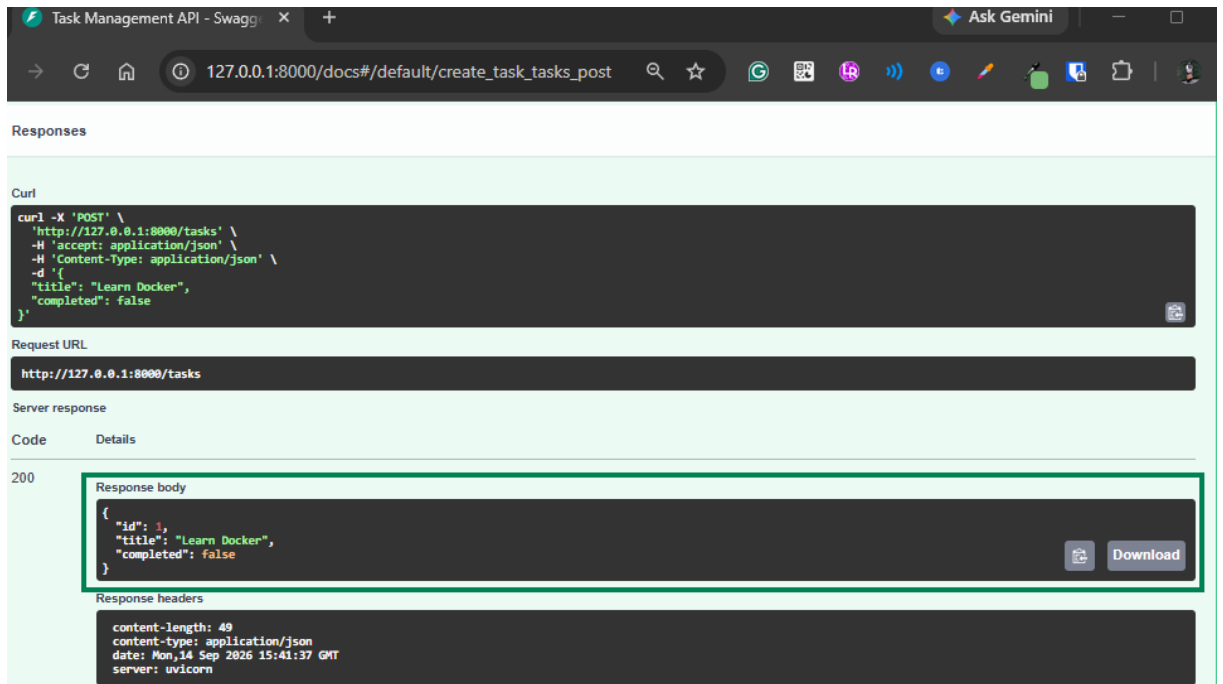
Click **Try it out** and enter:

```
{  
  "title": "Learn Docker",  
  "completed": false  
}
```

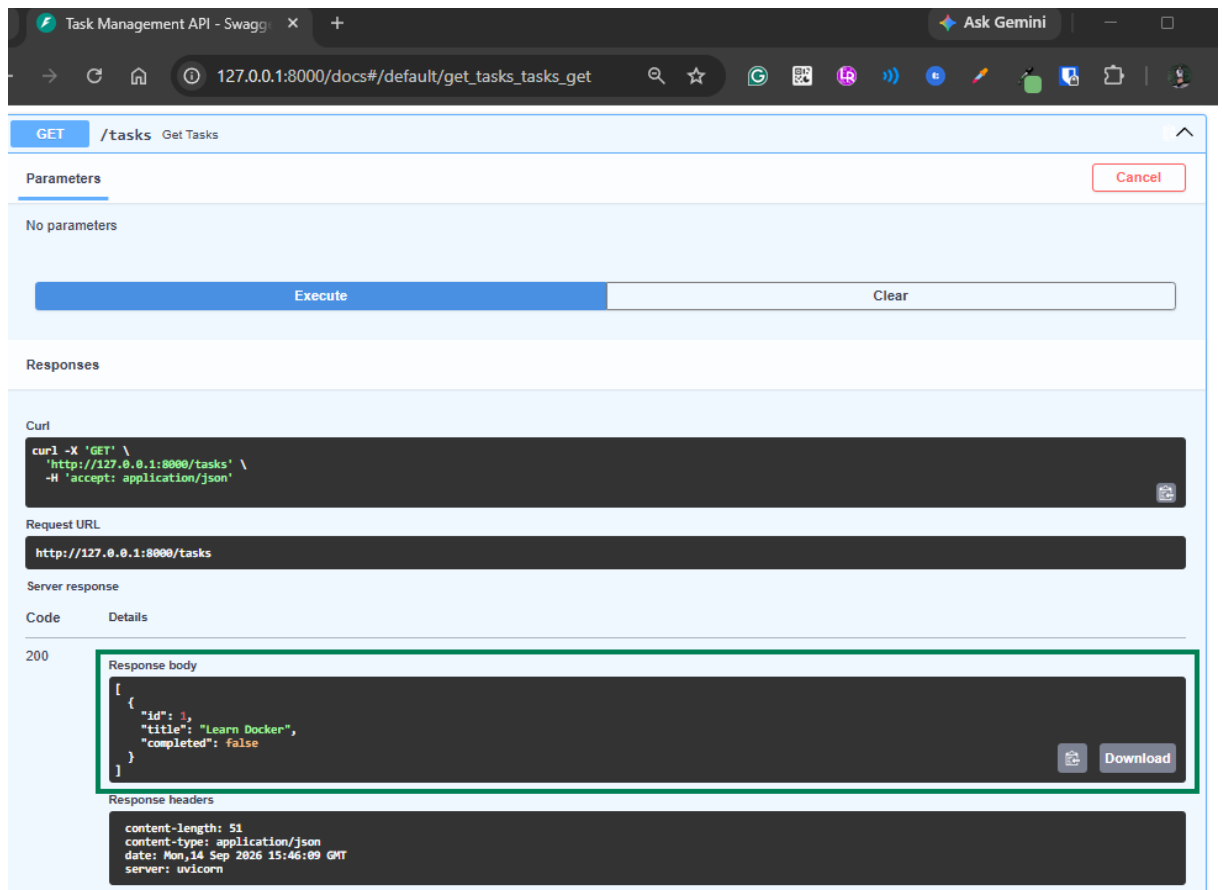


Click **Execute**.

Response:



And then.. try **GET /tasks**



Now... we already have REST API 😊

8. Dockerize

Now, we move on to DevOps.

Our application that we made now:

[Python](#)



[FastAPI](#)



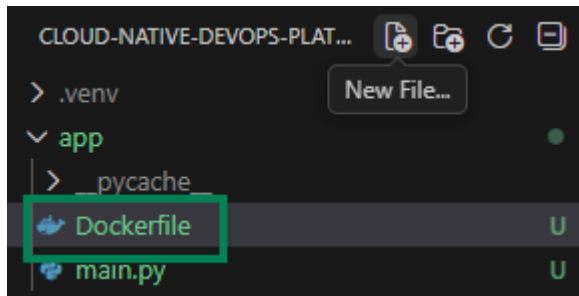
[Works on my laptop](#)

The problem: If it is moved to the server, will the environment be the same?

Docker helps to solve that problem.

Now, make Dockerfile

Inside app/Dockerfile



Write:

FROM python:3.12-slim

WORKDIR /app

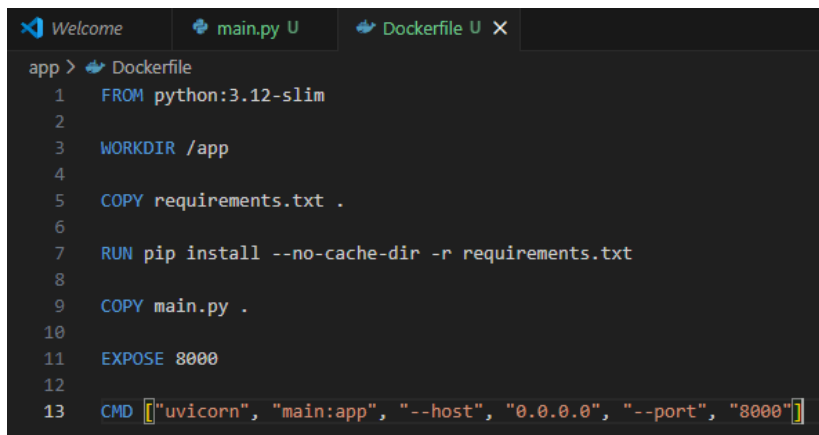
COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY main.py .

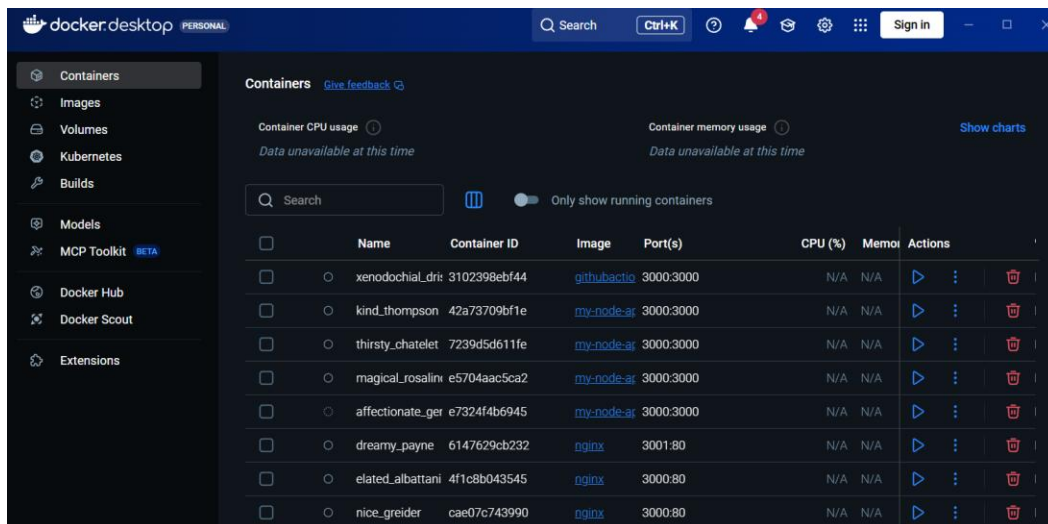
EXPOSE 8000

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]



9. Build Docker Image

Make sure Docker Desktop already running!



From **app** folder:

1. `docker build -t task-management-api .`
2. `docker images`

and then we should see **task-management-api**

```
PS C:\Users\Valin\Downloads\cloud-native-devops-platform\app> docker build -t task-management-api .
[+] Building 75.9s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 260B
=> [internal] load metadata for docker.io/library/python:3.12-slim
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/5] FROM docker.io/library/python:3.12-slim@sha256:78387bc3881b8273120a12ebe6c1ab22b018ccc2c9adf565a1ac9b
=> => resolve docker.io/library/python:3.12-slim@sha256:78387bc3881b8273120a12ebe6c1ab22b018ccc2c9adf565a1ac9b5
=> => sha256:56235e4245636e401a28cf88944d543b29ee028ae3b6c27e03d6b43e7b4eac5f 249B / 249B
=> => sha256:1560cfed01dc4c72c07f789d860078d74466a1f9a26b33a2e35d887b5f90dd3f 12.12MB / 12.12MB
=> => sha256:dbd0b7849e6c06b61e1fa6b7ffe053f246ce0075890620e3db64b47bb662433a 4.27MB / 4.27MB
=> => sha256:6310eb16bf4251731feab01e8f633bf5e2d75a657ccad97f420b1f83cce457be 29.79MB / 29.79MB
=> => extracting sha256:6310eb16bf4251731feab01e8f633bf5e2d75a657ccad97f420b1f83cce457be
=> => extracting sha256:dbd0b7849e6c06b61e1fa6b7ffe053f246ce0075890620e3db64b47bb662433a
=> => extracting sha256:1560cfed01dc4c72c07f789d860078d74466a1f9a26b33a2e35d887b5f90dd3f
=> => extracting sha256:56235e4245636e401a28cf88944d543b29ee028ae3b6c27e03d6b43e7b4eac5f
=> [internal] load build context
=> => transferring context: 650B
```

```
PS C:\Users\Valin\Downloads\cloud-native-devops-platform\app> docker images
REPOSITORY          TAG          IMAGE ID          CREATED          SIZE
task-management-api  latest      77ce88add46e     About a minute ago 238MB
gitlab-cicd         latest      d5fc6bfabb4f     7 months ago    192MB
hello-node          latest      e4d6439eb15e     7 months ago    192MB
```

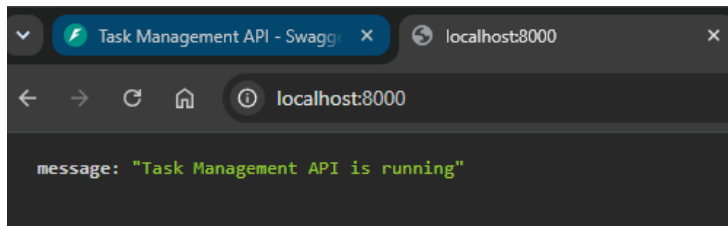
10. Run the container

`docker run -d -p 8000:8000 --name task-api task-management-api`

and now open <http://localhost:8000>

```
MINGW64/c/Users/Valin

Valin@DESKTOP-VMJ5NLD MINGW64 ~
$ docker run -d -p 8000:8000 --name task-api task-management-api
65d1b9aa870bd700906992f09c2533aa657695b472fd5f22a26f587d0ed6a3e
```



So, we just running the application in Docker

11. Test container

docker ps

```
va11m9@DESKTOP-VMJ5NLD MINGW64 ~  
$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
65d1b9aa870b	task-management-api	"uvicorn main:app --..."	2 minutes ago	Up 2 minutes	0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp	task-api
8946a792e8d4	mariadb:10.5	"docker-entrypoint.s..."	9 months ago	Up 24 minutes (healthy)	0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp	mariadb

It means,

Laptop

|
| port 8000



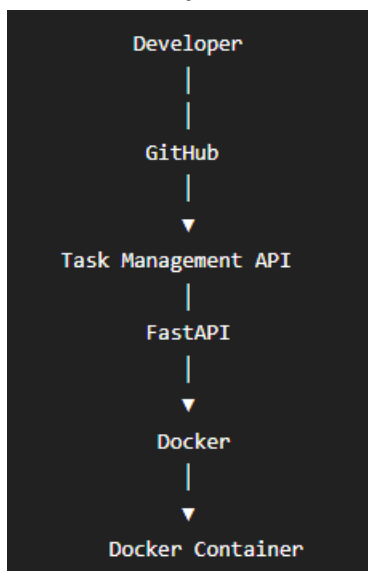
Docker Container

|



FastAPI

This is the day 1. So far we made:



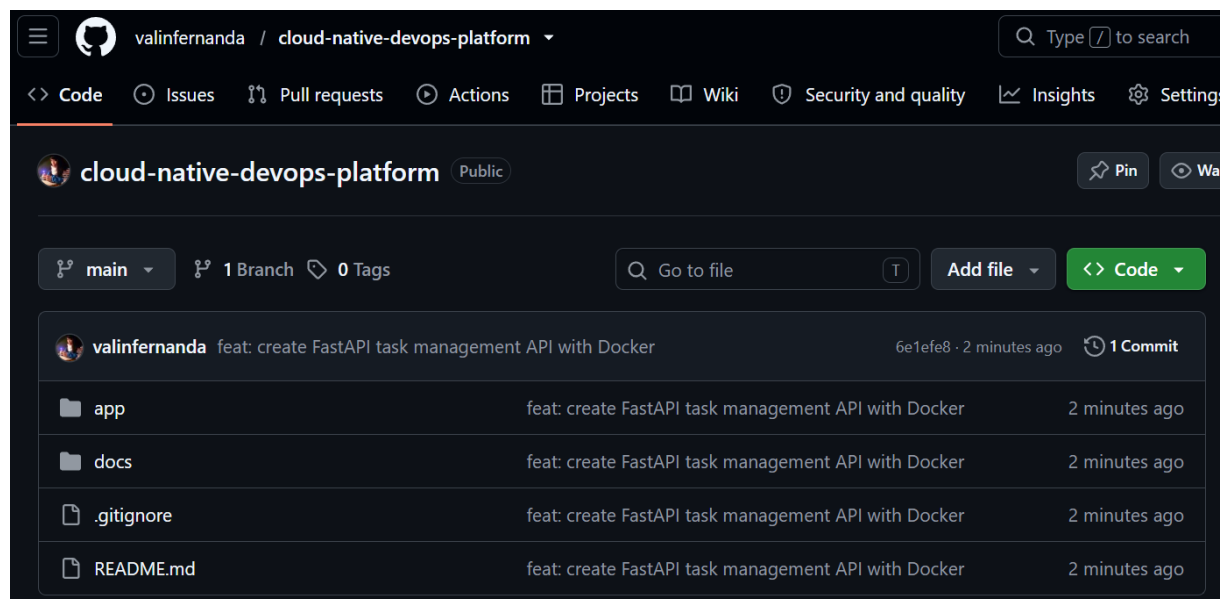
Push to Github

```
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore
        README.md
        app/
        docs/

nothing add to commit; use "git add" to track
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> git add .
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> git commit -m "feat: create FastAPI task management API with Docker"
[main (root-commit) 6e1efe8] feat: create FastAPI task management API with Docker
6 files changed, 147 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 README.md
 create mode 100644 app/Dockerfile
 create mode 100644 app/main.py
 create mode 100644 app/requirements.txt
 create mode 100644 docs/deployment.md
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> git remote add origin https://github.com/valinfernanda/cloud-native-devops-platform.git
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> git push -u origin main
Enumerating objects: 10, done.
Counting objects: 100% (10/10), done.
Delta compression using up to 12 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (10/10), 2.16 KiB | 441.00 KiB/s, done.
Total 10 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/valinfernanda/cloud-native-devops-platform.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```



valinfernanda / cloud-native-devops-platform

Code Issues Pull requests Actions Projects Wiki Security and quality Insights Settings

cloud-native-devops-platform Public

main 1 Branch 0 Tags

Go to file Add file Code

valinfernanda feat: create FastAPI task management API with Docker 6e1efe8 · 2 minutes ago 1 Commit

app	feat: create FastAPI task management API with Docker	2 minutes ago
docs	feat: create FastAPI task management API with Docker	2 minutes ago
.gitignore	feat: create FastAPI task management API with Docker	2 minutes ago
README.md	feat: create FastAPI task management API with Docker	2 minutes ago

Self reflection....

1. What is an API?
2. What is the difference between GET and POST?
3. What is FastAPI?
4. What is Docker?
5. What is the difference between a Docker image and a Docker container?
6. Why do our applications need to be containerized?
7. Why does our DevOps project require an application?